

بسمه تعالی

نیازمندیهای عملکردی

شناسایی و مدلسازی

نیازمندی در دنیای نرم افزار = Requirements

- نیازمندی به زبان ساده: «چیزی است که سیستم نرم افزاری باید انجام دهد یا ویژگی ای که باید داشته باشد».

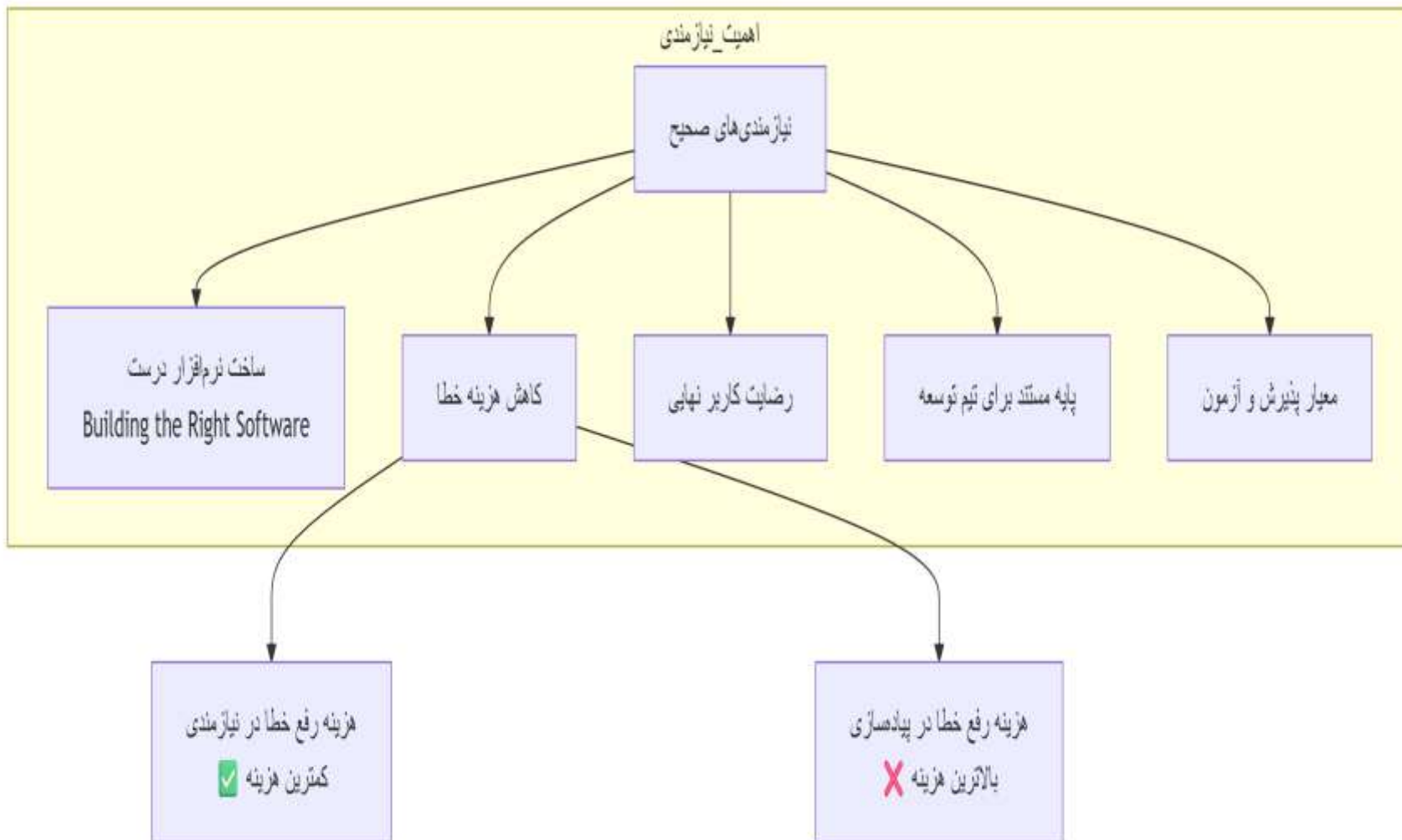
نیازمندی عبارت است از:

- شرط یا قابلیت که یک کاربر برای حل یک مسئله یا رسیدن به یک هدف به آن نیاز دارد.
- شرط یا قابلیت که یک سیستم باید آن را برآورده سازد یا داشته باشد.
- نمایش مستند از یک شرط یا قابلیت فوق.

انواع نیازمندیها

تعریف یک خطی	نوع نیازمندی
عملی که سیستم در پاسخ به ورودی خاص باید انجام دهد.	نیازمندی کارکردی (Functional Requirement)
معیار کیفی یا معیار عملکردی که سیستم باید برآورده کند.	نیازمندی غیرکارکردی (Nonfunctional Requirement)
قید فنی، سازمانی یا محیطی که آزادی عمل طراح را محدود می کند.	محدودیت (Constraint)

نمونه	نوع
سیستم باید امکان جستجوی کتاب بر اساس عنوان را فراهم کند.	کارکردی
جستجو باید حداکثر در ۲ ثانیه نتیجه را برگرداند.	غیرکارکردی
سیستم باید با پایگاه داده موجود سازمان یکپارچه شود.	محدودیت



تعریف سیستم براساس نیازمندیها



یک سیستم نرم افزاری چیزی جز «پاسخ به مجموعه‌ای از نیازمندی‌ها» نیست.

■ نیازمندی پل ارتباطی بین کاربر (که مسئله را می‌داند) و تیم فنی (که راه حل می‌سازد) است.

■ سیستم نرم افزاری دقیقاً همان چیزی است که نیازمندی‌هایش می‌گویند. نه بیشتر، نه کمتر.

سیستم = مجموع نیازمندی‌های کارکردی +

مجموع نیازمندی‌های غیرکارکردی

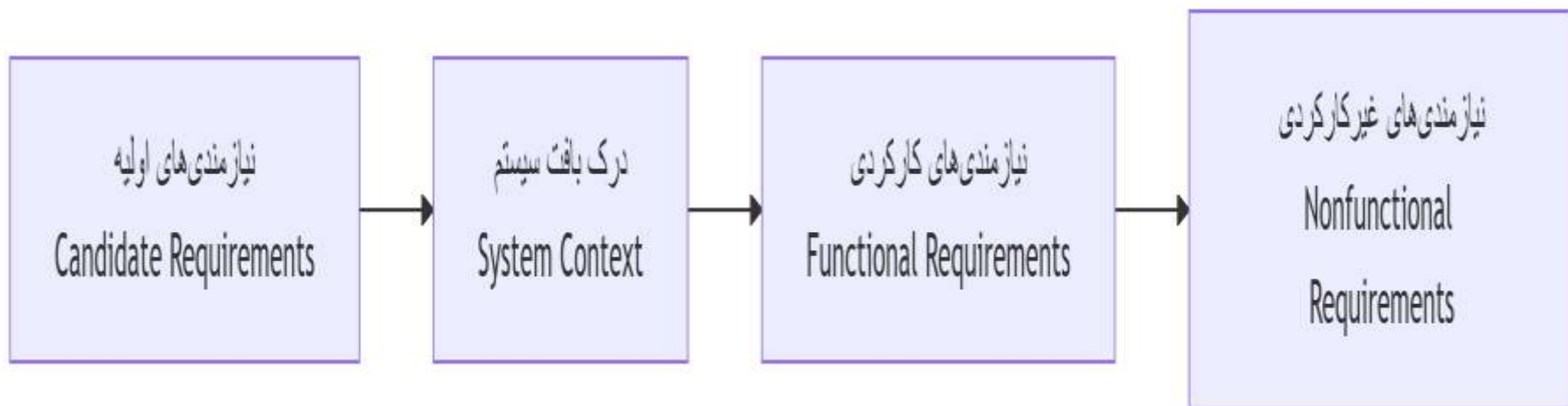
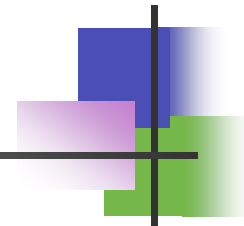
(که به درستی پیاده‌سازی شده‌اند)

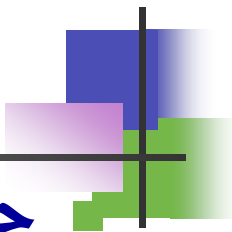
■ فرد بروکس: سخت‌ترین بخش ساخت یک سیستم نرم افزاری، دقیقاً تصمیم‌گیری درباره «چه چیزی ساخته شود» است.

گردش کاری نیازمندیها

: Requirements Workflow

- به مجموعه‌ای از فعالیت‌های ساختاریافته و گام‌به‌گام گفته می‌شود که طی آن، نیازمندی‌های یک سیستم نرم‌افزاری از حالت ایده‌آل و مبهم به مستندات شفاف، دقیق و قابل پیاده‌سازی تبدیل می‌شود.
- به زبان ساده‌تر: فرآیندی که به ما می‌گوید: «چه چیزی باید بسازیم؟» قبل از اینکه بپرسیم «چگونه بسازیم؟»





چهار مرحله اصلی برای بدست آوردن نیازمندیها

این چهار مرحله به ترتیب انجام میشود :

- بدست آوردن لیست از نیازمندیهای اولیه

List candidate requirements

- درک زمینه و بافت سیستم

Understand system context

- بدست آوردن نیازمندیهای کارکردی

Capture functional requirements

- بدست آوردن نیازمندیهای غیرکارکردی

Capture nonfunctional requirements

مرحله اول : بدست آوردن لیست اولیه نیازمندیها

هدف بدست آوردن قابلیت ها (Features)

- منظور یک نیازمندی اولیه با یک تعریف کلی که هنوز تصمیمی برای اینکه به یک نیازمندی تبدیل شود، گرفته نشده است.
- لیست Feature ها ساخته میشود (در جلسات Brain Storming) و بدون بحث در مورد قبول یا رد آنها
 - کاربران، مشتریان، تحلیل گران و توسعه دهندگان در این جلسات حضور دارند و نظر میدهند
 - طبقه بندی میشود کرد ولی بحث نداریم
- تهیه توضیح و یک تعریف برای هر Feature
 - ممکن است در مراحل بعدی پروژه برای یک Feature تصمیم گیری شود(تبدیل به نیاز شود – محقق شود)
 - ممکن است هیچ گاه به یک Feature ارجاع نشود
 - یک لیست (شناسنامه) از Feature ها خواهیم داشت که در طی پروژه به آن ارجاع میشود



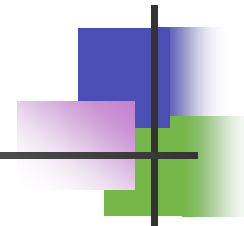
جلسه طوفان فکری با حضور
کاربران، مشتریان، تحلیلگران و
توسعه دهندگان

گام ۱: هرکس هر ایده‌ای دارد
می‌گوید - بدون بحث در مورد قبول
یا رد

گام ۲: تهیه لیست قابلیت‌ها -
طبقه‌بندی می‌شود، ولی بحث نداریم

گام ۳: تهیه توضیح و یک تعریف
برای هر قابلیت

گام ۴: ایجاد شناسنامه قابلیت‌ها برای
ارجاع در طول پروژه



قانون	توضیح
بدون بحث در مورد قبول یا رد	در این مرحله فقط جمع آوری می کنیم، قضاوت نمی کنیم
همه ذی نفعان حضور دارند	کاربر، مشتری، تحلیلگر، توسعه دهنده - همه نظر می دهند
طبقه بندی مجاز، بحث ممنوع	می توان دسته بندی کرد ولی در مورد ارزش یا امکان پذیری بحث نمی کنیم
هر Feature یک توضیح و تعریف دارد	شفافیت برای ارجاعات بعدی

Features List

وضعیت Feature «

پیشنهاد شده

تایید شده

در حال انتظار

رد شده

....

تخمین هزینه : براساس پارامترهایی نظیر نفر- ساعت یا بصورت point نظیر

(Story Point) Scrum

افسانه نفر ساعت (بروکس)

فرمول لشکر مغول (Mongolian horde formula)

سطح Risk پیاده سازی

اولویت

Critical

important

optional

Features List



شناسه	نام قابلیت	توضیحات	وضعیت	تخمین هزینه (نفر-ساعت)	ریسک	اولویت
F-۲۰۱	ورود به سیستم	کاربر با نام کاربری و رمز عبور وارد شود	تأیید شده	۱۶	کم	بحرانی
F-۲۰۲	مشاهده دوره‌ها	دانشجو بتواند لیست دوره‌های خود را مشاهده کند	تأیید شده	۸	کم	بحرانی
F-۲۰۳	دانلود محتوا	دانشجو بتواند فایل‌های آموزشی را دانلود کند	تأیید شده	۱۲	کم	بحرانی
F-۲۰۴	آپلود تمرین	دانشجو بتواند تمرین خود را آپلود کند	تأیید شده	۲۴	متوسط	مهم
F-۲۰۵	برگزاری آزمون آنلاین	استاد بتواند آزمون آنلاین طراحی و برگزار کند	در حال انتظار	۴۰	زیاد	مهم
F-۲۰۶	نمره‌دهی خودکار	سیستم به سوالات تستی نمره بدهد	پیشنهاد شده	۳۲	زیاد	اختیاری
F-۲۰۷	گواهی پایان دوره	پس از اتمام دوره، گواهی به صورت خودکار صادر شود	پیشنهاد شده	۲۰	متوسط	اختیاری

درک بافت سیستم

(Understand System Context)

- پس از گردآوری فهرست اولیه ویژگی‌ها در جلسات طوفان فکری، گام اساسی بعدی درک بافت سیستمی است که قصد ساخت آن را داریم.
- نظریه‌پردازان مهندسی نیازمندی‌ها: "هدف اصلی در یک مسئله توسعه نرم‌افزار، ایجاد نرم‌افزاری است که هدفی مفید را در جهان بیرون برآورده سازد". (به یک نیاز واقعی پاسخ دهد- خلق ارزش کند)
- توسعه‌دهندگان نرم‌افزار معمولاً با مسئله‌ای بلافاصله قابل تشخیص و کاملاً درک شده مواجه نیستند، بلکه باید با پرسش از "این مسئله از چه نوعی است؟"، "دقیقاً مسئله درباره چیست؟" و "جهان بیرون چه هدفی را دنبال می‌کند؟" آغاز کنند.
- در این مرحله، هدف «تحلیل دامنه» (Domain Analysis) است.
- روش FODA (Feature-Oriented Domain Analysis) که در پژوهش‌های آکادمیک به کار رفته، بر ایجاد «نمودار ویژگی‌ها» برای توصیف دامنه مسئله تأکید دارد. این روش به ما امکان می‌دهد تا ویژگی‌های اولیه را در ساختاری منسجم، سازماندهی کرده و ارتباط میان مفاهیم کلیدی دامنه را آشکار سازیم.

درک بافت سیستم

(Understand System Context)



"درک بافت سیستم" به معنای درک محیطی است که سیستم قرار است در آن کار کند.

بسیاری از افراد درگیر در توسعه نرم افزار، متخصصان مسائل نرم افزاری هستند. با این حال، برای استخراج نیازمندی‌های صحیح و ساخت سیستم درست، توسعه‌دهندگان کلیدی - به ویژه معمار و برخی از تحلیلگران ارشد - نیاز به درک محکمی از بافتی دارند که سیستم در آن قرار می‌گیرد.

هدف از درک بافت سیستم، کمک به درک مسئله‌ای است که سیستم قرار است در رابطه با بافت خود حل کند.

مدل‌سازی دامنه باید به درک مسئله کمک کند، نه راه‌حل داخلی سیستم. نحوه حل مسئله توسط سیستم در مراحل تحلیل، طراحی و پیاده‌سازی بررسی خواهد شد.

درک بافت سیستم

(Understand System Context)

بر اساس متن رسمی Unified Process، حداقل دو رویکرد برای بیان بافت سیستم به شکلی قابل استفاده توسط توسعه‌دهندگان نرم‌افزار وجود دارد:

- **مدل‌سازی دامنه (Domain Modeling):** مدل دامنه مهم‌ترین انواع اشیاء را در بافت سیستم ثبت می‌کند. اشیاء دامنه، "چیزهایی" را نشان می‌دهند که در محیطی که سیستم در آن کار می‌کند، وجود دارند یا رویدادهایی که رخ می‌دهند.
- **مدل‌سازی کسب‌وکار (Business Modeling):** مدل کسب‌وکار را می‌توان به عنوان ابر مجموعه‌ای از مدل دامنه توصیف کرد که فراتر از اشیاء دامنه است. هدف مدل‌سازی کسب‌وکار، توصیف فرآیندها (موجود یا درک شده) به منظور درک آنهاست.
- مدل کسب‌وکار مشخص می‌کند که کدام فرآیندهای کسب‌وکار باید توسط سیستم پشتیبانی شوند. علاوه بر شناسایی اشیاء کسب‌وکار یا دامنه درگیر در کسب‌وکار، مدل‌سازی کسب‌وکار همچنین شایستگی‌های مورد نیاز در هر فرآیند را تعیین می‌کند: کارگران، مسئولیت‌های آنها و عملیاتی که انجام خواهند داد.

درک بافت سیستم

چه رویکردی؟

رویکرد ساده

مدل دامنه

زیر مجموعه

رویکرد کامل

مدل کسب و کار

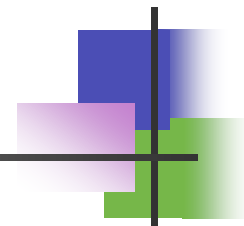
فقط اشیاء و مفاهیم

مناسب برای سیستم‌های اطلاعاتی

فرآیندها + اشیاء + بازیگران +

مناسب برای سیستم‌های فرآیندمحور

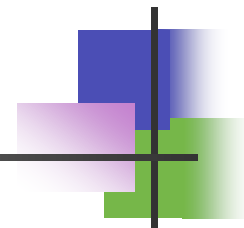
Understand system context



مدل دامنه Domain Model

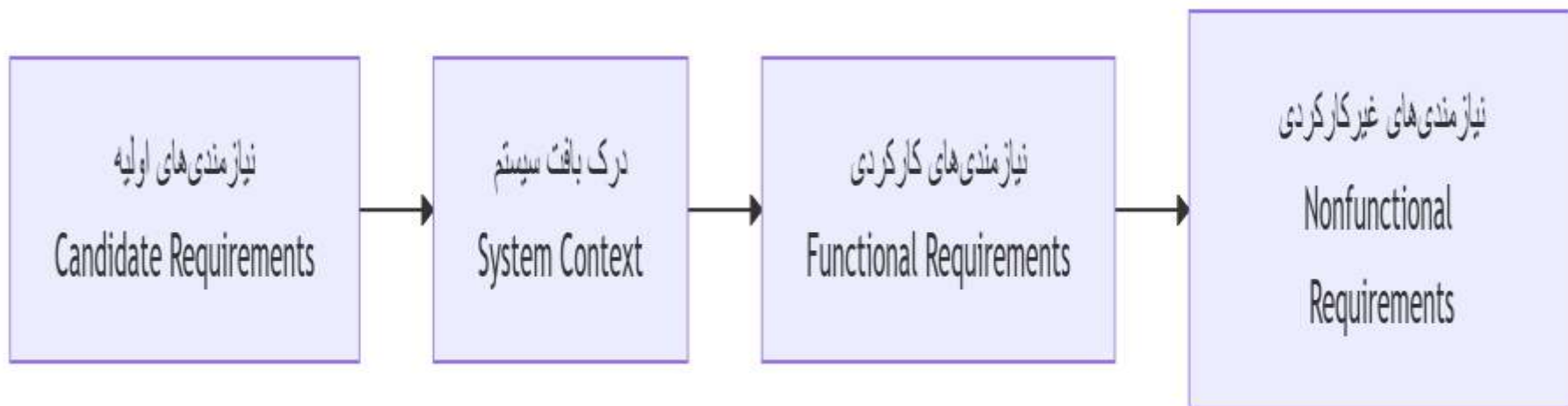
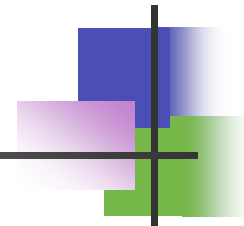
- مدل ساختاري
- معرفي دقيق بافت و context
- شناسايي مفاهيم مهم در بافت و بيان ارتباط بين آنها = حرکت به سمت کلاس دیاگرام
- تشکیل واژه نامه و ترمینولوژی (Glossary of Terms) -
 - اهمیت بالای تعريف دقيق اصطلاحات و ارتباط بين آنها
 - زبان مشترك بين شما و کاربر (کاربر يك متخصص در حوزه است و شما بایستی با اصطلاحات حوزه آشنا شوید و وارد فضاي کسب و کار شوید)
 - انجام سایر امور شناخت نظیر مصاحبه بر این اساس (اگر زبان مشترك نداشته باشیم، مصاحبه کارایی ندارد)
- شناسايي Domain object که اشیاء اصلي دامنه مسئله هستند و رسم Class diagram
- کلاسهاي دامنه الزاما تبدیل به کلاس سيستمي نخواهند شد ولي در شناخت مسئله کمک میکند
 - مثال) در يك نظام آموزش با ثبت نام دستی : کارمند آموزش که فرمهاي ثبت نام را قبل از درج در سيستم کنترل میکند(در سيستم دستی) و این کار در يك سيستم کامپیوتري موضوعیت ندارد و عملا این شي دامنه تبدیل به کلاس نمیشود.
- شناسايي سيستم و زیرسيستم هاي آن براساس شناخت حاصله
 - عملا با استفاده از package diagram، زیر سيستم ها معرفي شده و هر package از تعدادي کلاس(سلسله مراتب)

Understand system context



■ مدل کسب و کار Business Model

- مدل رفتاری
- مدل سازی فعالیتهای کسب و کار سازمان (در حوزه مربوطه)
- چه کارهای در چه واحدهایی و توسط چه افرادی انجام میشود
- Activity Diagram
- از بین فعالیتهای و فرآیندهای شناسایی شده، کدامیک قرار است توسط نرم افزار مورد نظر پیاده سازی شود (پشتیبانی شود)



Capture functional requirements



تعریف نیازمندی‌های عملکردی (Functional Requirements) :

- نیازمندی‌های عملکردی مشخص می‌کنند که سیستم چه خدمات، رفتارها و قابلیت‌هایی باید ارائه دهد و سیستم در پاسخ به ورودی‌ها، رویدادها و تعاملات کاربران چگونه عمل می‌کند.
- این نیازمندی‌ها بیانگر وظایف قابل مشاهده سیستم از دید ذی‌نفعان هستند و معمولاً به صورت سناریو، سرویس، فرایند کسب‌وکار یا Use Case مدل می‌شوند.
- نیازمندی عملکردی معمولاً با Use Case نمایش داده می‌شود. هر Use Case نماینده یک خدمت (Service) است که سیستم به یک Actor ارائه می‌کند.
- سؤال کلیدی برای کشف آن: «سیستم باید چه کاری انجام دهد؟»
- مثال‌ها کاربر بتواند
- وارد سیستم شود.
- سیستم امکان ثبت سفارش را فراهم کند
- سیستم گزارش فروش ماهانه تولید کند.
- مشتری بتواند وضعیت سفارش خود را مشاهده کند.

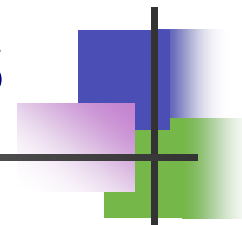
Capture functional requirements



تعریف نیازمندی‌های عملکردی (Functional Requirements) :

- نیازمندی‌های عملکردی مشخص می‌کنند که سیستم چه خدمات، رفتارها و قابلیت‌هایی باید ارائه دهد و سیستم در پاسخ به ورودی‌ها، رویدادها و تعاملات کاربران چگونه عمل می‌کند.
- این نیازمندی‌ها بیانگر وظایف قابل مشاهده سیستم از دید ذی‌نفعان هستند و معمولاً به صورت سناریو، سرویس، فرایند کسب‌وکار یا Use Case مدل می‌شوند.
- نیازمندی عملکردی معمولاً با Use Case نمایش داده می‌شود. هر Use Case نماینده یک خدمت (Service) است که سیستم به یک Actor ارائه می‌کند.
- سؤال کلیدی برای کشف آن: «سیستم باید چه کاری انجام دهد؟»
- مثال‌ها کاربر بتواند
 - وارد سیستم شود.
 - سیستم امکان ثبت سفارش را فراهم کند
 - سیستم گزارش فروش ماهانه تولید کند.
 - مشتری بتواند وضعیت سفارش خود را مشاهده کند.
 - صورتحساب خود را از سیستم دریافت کند
 - نسبت به مغایرت مالی خود ثبت اعتراض کند

Capture functional requirements



تعریف نیازمندی‌های غیرعملکردی (NON Functional Requirements) :

نیازمندی‌های غیرعملکردی ویژگی‌ها، محدودیت‌ها و معیارهای کیفیتی سیستم را مشخص می‌کنند و بیان می‌کنند که سیستم چگونه باید عملکردهای خود را ارائه کند، نه اینکه چه کاری انجام دهد. این نیازمندی‌ها معمولاً به ویژگی‌هایی مانند:

- Performance (کارایی): زمان پاسخ، سرعت پردازش
- Reliability (قابلیت اطمینان): پایداری، نرخ خطا
- Security (امنیت): کنترل دسترسی، محرمانگی
- Usability (کاربرپسندی): سهولت استفاده
- Scalability (مقیاس‌پذیری)
- Availability (دسترس‌پذیری)
- Maintainability (نگهداشت‌پذیری)

سؤال کلیدی برای کشف آن: «سیستم باید چگونه این کار را انجام دهد؟»

- زمان پاسخ سیستم نباید بیش از ۲ ثانیه باشد.
- فقط کاربران احراز هویت‌شده مجاز به ثبت سفارش باشند.
- سیستم باید ۹۹,۹٪ در دسترس باشد.
- سامانه باید همزمان از ۱۰۰۰ کاربر پشتیبانی کند.

Capture functional requirements

Capture functional requirements

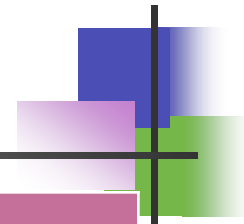
نیازمندیهای عملکردی با استفاده از یوزکیس نمایش داده میشود

- هر یوزکیس توصیف کننده روش استفاده از سیستم توسط یک کاربر است
- هر یوزکیس متناظر است با یک سرویسی که کاربر از سیستم میگیرد

مجموعه یوزکیس ها == مجموعه سرویسهایی که سیستم ارائه میکند

مدل Use Case شامل

- همه یوزکیس ها به همراه Use case diagram
- توصیف یوزکیس ها
- استفاده از گلاسوری اصطلاحات (که در مرحله قبل و مدلسازی دامنه ساخته شده است)
- پروتوتایپ UI که بر اساس یوزکیس ها ساخته میشود
- سه اصطلاح مهم که طرف مصاحبه به منظور جمع آوری اطلاعات میشوند
 - User « استفاده کننده از سیستم (محصول پروژه)
 - Customer « سفارش دهنده پروژه
 - Sponsor « تامین کننده مالی پروژه



نیازمندی عملکردی	نیازمندی غیرعملکردی
سیستم چه کاری انجام می دهد	سیستم چگونه آن کار را انجام می دهد
رفتار و سرویس سیستم	کیفیت و محدودیت سیستم
معمولاً با Use Case مدل می شود	معمولاً به صورت Constraint یا Quality Attribute بیان می شود
مثال: ثبت سفارش	مثال: ثبت سفارش در کمتر از ۳ ثانیه

Use Case Model

- پس از شناسایی نیازمندی‌های اولیه و درک بافت سیستم، برای مدل‌سازی نیازمندی‌های عملکردی از Use Case استفاده می‌کنیم.
- زیرا هر Use Case نماینده یک خدمت است که سیستم به بازیگران بیرونی ارائه می‌دهد.
- در رویکردهای سنتی تحلیل سیستم، نیازمندی‌ها غالباً به صورت مجموعه‌ای از قابلیت‌ها، وظایف و محدودیت‌ها در قالب فهرستی از Requirements مستندسازی می‌شدند. در این شیوه، تمرکز اصلی بر «آنچه سیستم باید انجام دهد» قرار داشت و نیازمندی‌ها معمولاً به صورت مستقل و جدا از بافت تعامل کاربران با سیستم بیان می‌شدند.
- مصاحبه با ذینفعان کلیدی سامانه
 - کاربران (محوری ترین ذینفع سیستم)
 - مشتریان سیستم
 - اسپانسر سامانه (که ممکن است همان مشتری باشد)
- سطوح برگزاری مصاحبه بر اساس نقش‌های آنها
- نیازمندی‌هایی که در این مرحله صورت می‌گیرد، در هر لحظه امکان تغییر وجود دارد
- امکان تغییر نیازمندی در هر زمانی بایستی توسط ذینفعان سامانه امکان پذیر باشد (حتی در انتهای پروژه)
- شیوه‌های چابک شعار اصلی آنها پذیرش تغییر است
- اصالت بر رضایت مشتری است تا قرارداد و متن آن
- شعار XP: استقبال از تغییر در هر زمان و هر موقعیت از پروژه

Use Case Model



- پس از شناسایی نیازمندی‌های اولیه و درک بافت سیستم، برای مدل‌سازی نیازمندی‌های عملکردی از Use Case استفاده می‌کنیم.
- زیرا هر Use Case نماینده یک خدمت است که سیستم به بازیگران بیرونی ارائه می‌دهد.
- در رویکردهای سنتی تحلیل سیستم، نیازمندی‌ها غالباً به صورت **مجموعه‌ای از قابلیت‌ها، وظایف و محدودیت‌ها در قالب فهرستی از Requirements مستندسازی** می‌شدند.
- در این شیوه، تمرکز اصلی بر **«آنچه سیستم باید انجام دهد»** قرار داشت و نیازمندی‌ها معمولاً به صورت مستقل و جدا از بافت تعامل کاربران با سیستم بیان می‌شدند.
- در مقابل، در تحلیل و طراحی شی‌گرا، به‌ویژه در رویکرد مبتنی بر Use Case که توسط یاکوبسن مطرح شد، نیازمندی‌های عملکردی نه به صورت یک فهرست منفصل، بلکه بر اساس **تعامل بازیگران بیرونی یا آکتور با سیستم مدل** می‌شوند.
- در این رویکرد، برای هر دسته از کاربران هم‌نقش یا هم‌هدف، مجموعه‌ای از خدمات و تعاملات مورد انتظار سیستم در قالب Use Case بیان می‌شود.

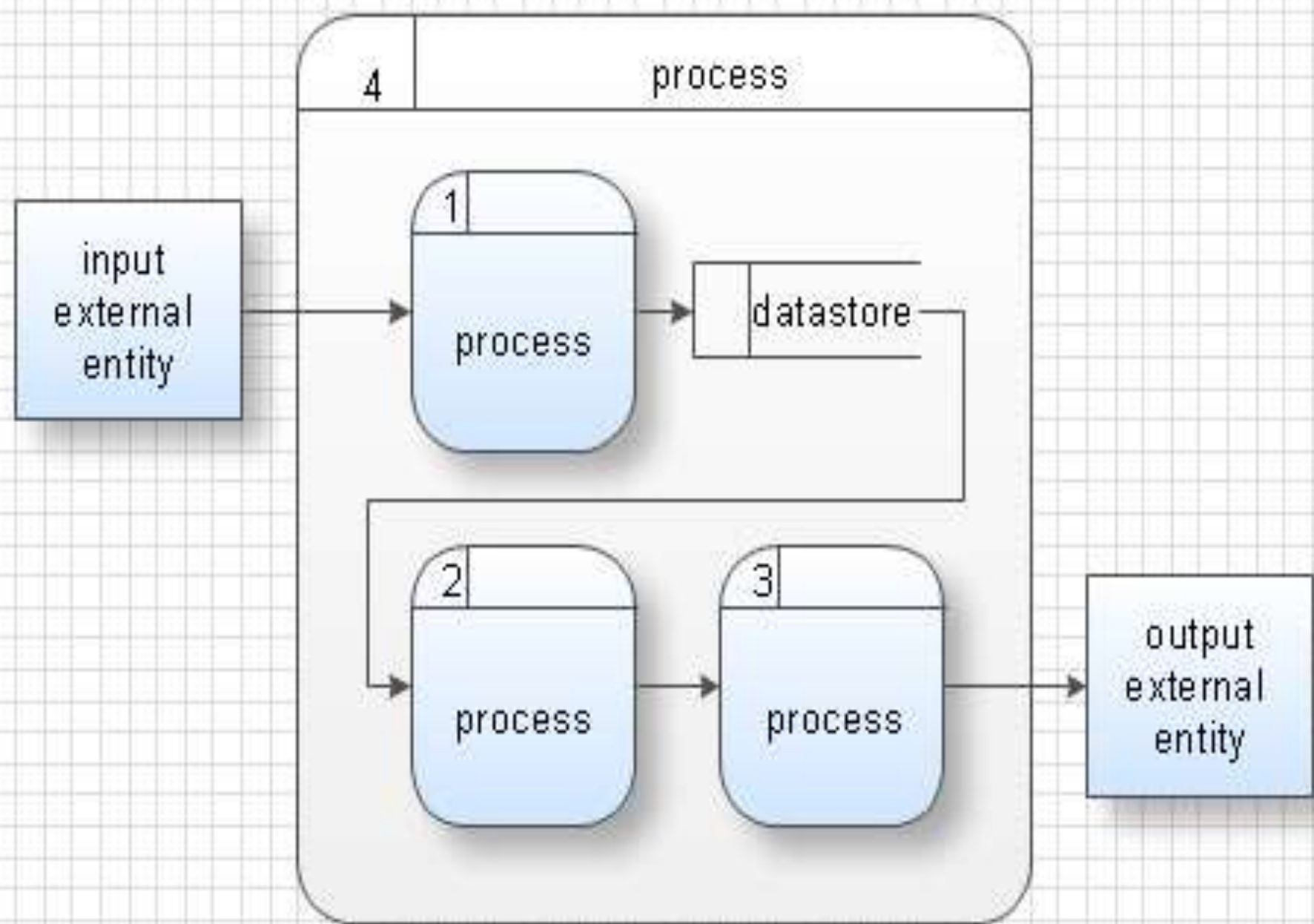
Use Case Model

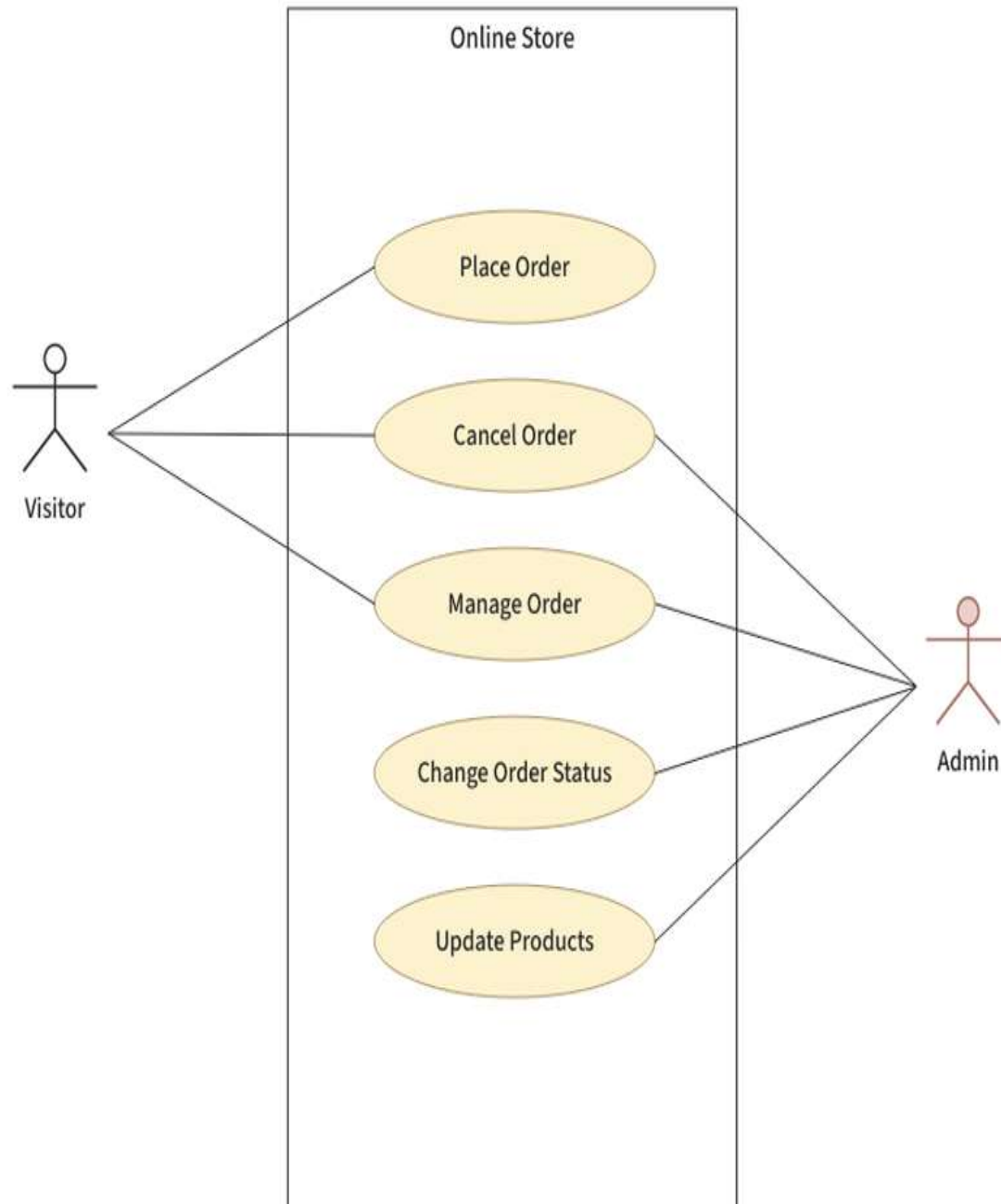
■ به بیان دقیق‌تر، Use Case توصیف‌کننده‌ی یک تعامل معنادار میان یک Actor و سیستم است که به دستیابی به هدفی مشخص و ایجاد نتیجه‌ای دارای ارزش برای Actor منجر می‌شود.

■ از این منظر، هر Use Case نماینده‌ی یک خدمت یا رفتار قابل مشاهده‌ای است که سیستم در پاسخ به نیاز یک بازیگر بیرونی ارائه می‌کند.

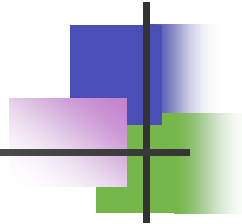
■ بنابراین، در طراحی شی‌گرا، نیازمندی‌های عملکردی عمدتاً از دیدگاه کاربران و در قالب سناریوهای استفاده از سیستم مدل‌سازی می‌شوند، نه صرفاً به صورت فهرستی از قابلیت‌ها

■ .



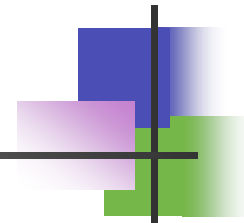


Use Case vs. Use Case Model vs. Use Case Diagram



- **Use Case**: توصیف‌کننده‌ی یک تعامل معنادار میان Actor و سیستم برای دستیابی به یک هدف مشخص است و یک سرویس یا رفتار قابل مشاهده‌ی سیستم را از دید کاربر نمایش می‌دهد.
- **Use Case Diagram**: یک نمودار رفتاری در UML است که Actor ها، Use Case ها و روابط میان آنها را به صورت تصویری نمایش می‌دهد و نمایی سطح بالا از نیازمندی‌های عملکردی سیستم ارائه می‌کند.
- **Use Case Model**: مجموعه‌ی سازمان‌یافته‌ی تمامی Use Case ها، Use Case Diagram ها، Actor ها، روابط و توصیفات متنی است که در مجموع، نیازمندی‌های عملکردی سیستم را مدل و مستندسازی می‌کند.

Use Case



- **Use Case** توصیف کننده‌ی یک تعامل معنادار میان یک **Actor** و سیستم برای دستیابی به یک هدف مشخص و دارای ارزش برای **Actor** است.
- هر یوزکیس یک **Business Goal** را محقق می‌کنند.
- هر **Use Case** نماینده‌ی یک سرویس (**Service**) یا رفتار قابل مشاهده سیستم است. یعنی سیستم از نگاه کاربر دیده می‌شود، نه از نگاه ساختار داخلی.
- **Use Case** یک هدف کاربرمحور (**Goal-Oriented**) و یک سرویس معنادار سیستم است، نه یک عمل ریز و منفرد.
- یعنی **Use Case** معمولاً: درشت‌دانه است، دارای سناریو و چندین گام تعامل است، و به یک نتیجه دارای ارزش برای **Actor** ختم می‌شود.
- هدف اصلی:
- توصیف رفتار سیستم از دید بازیگران بیرونی
- سؤال کلیدی:
- کاربر برای رسیدن به یک هدف چگونه از سیستم استفاده می‌کند؟

یوز کیس ثبت سفارش

ثبت سفارش

سناریو:

۱. مشتری کالا را انتخاب می کند.
۲. سیستم سبد خرید را نمایش می دهد.
۳. مشتری اطلاعات پرداخت را وارد می کند.
۴. سیستم سفارش را ثبت می کند.
۵. شماره سفارش تولید می شود.

- در تحلیل، گاهی یک Use Case را می توان به Use Case های کوچک تر شکست ولی باز هم هر جزء باید یک رفتار معنادار و reusable باشد، نه صرفاً یک operation سطح پایین.
- اتمیک بودن یوز کیس در سطح عملیاتی نیست؛ بلکه یک واحد هدف محور از تعامل آکتور با سیستم است که نتیجه ای دارای ارزش برای آکتور تولید می کند.
- Use Case یک واحد اتمیک در سطح هدف (Goal-Level) است، نه در سطح عملیات (Operation-Level)؛
- یعنی مجموعه ای از تعاملات مرتبط که یک ارزش مشخص برای Actor ایجاد می کند.

Use Case Diagram



■ Use Case Diagram یک نمودار رفتاری UML است که Use Case ها، Actor ها و روابط میان آنها را به صورت تصویری نمایش می دهد.

■ عناصر اصلی Use Case Diagram

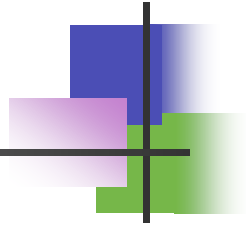
- Actor
- Use Case
- System Boundary
- Relationships
 - Association
 - Include
 - Extend
 - Generalization

Use Case Model



- **Use Case Model**: مجموعه‌ی کامل و سازمان‌یافته‌ای از تمام Use Case ها، Actor ها، Diagram ها و توصیفات متنی است که نیازمندی‌های عملکردی سیستم را مدل می‌کند.
- **نمای کلان (High-Level View)**: از نیازمندی‌های عملکردی ارائه می‌دهد.
- یعنی نشان می‌دهد:
 - چه Actor هایی وجود دارند؟
 - سیستم چه سرویس‌هایی دارد؟
 - چه کسی از چه سرویسی استفاده می‌کند؟
 - وابستگی‌ها چگونه‌اند؟
- اما توضیح نمی‌دهد:
 - ترتیب مراحل چیست؟
 - استثناها چیست؟
 - قواعد کسب‌وکار چیست؟

Use Case Model

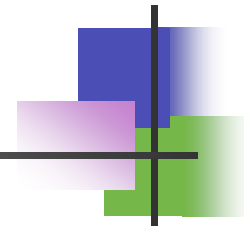


Use Case Model = Functional View of the System

Use Case Model اجزای ■

1. Use Case Diagram(s): نمای تصویری
2. Use Case Descriptions: توصیف سناریوها
3. Actors: بازیگران سیستم
4. Relationships: روابط UML
5. Glossary: واژگان دامنه
6. Special Requirements: Use Case نیازمندی‌های خاص هر
7. Supplementary Requirements: نیازمندی‌های مکمل

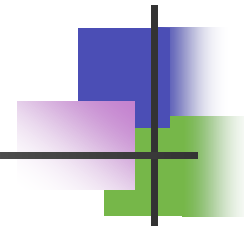
Capture NonFunctional Requirements(NFRs)



جایگاه نیازمندی های غیر عملکردی

- اگر Functional Requirement پاسخ دهد:
سیستم چه کاری انجام می دهد؟
- Non-Functional Requirement پاسخ می دهد:
سیستم آن کار را چگونه، با چه کیفیتی و تحت چه محدودیت هایی انجام می دهد؟
NFRها رفتار سیستم را تغییر نمی دهند، بلکه کیفیت تحقق رفتار را مشخص می کنند.
- مثال:
 - Functional Requirement
کاربر بتواند سفارش ثبت کند.
 - Non-Functional Requirement
ثبت سفارش باید در کمتر از ۲ ثانیه انجام شود.
 - یا: فقط کاربران احراز هویت شده مجاز به ثبت سفارش باشند.

Capture NonFunctional Requirements(NFRs)



- **قابلیت اطمینان (Reliability):** میزان توانایی سیستم در ارائه عملکرد صحیح، پایدار و بدون خطا در یک بازه زمانی مشخص.
- سیستم تا چه میزان قابل اعتماد است و با چه نرخ پائینی دچار خطا می شود؟
- معیارها:
 - نرخ خرابی پایین (Failure Rate)
 - فاصله زمانی میان دو خرابی (MTBF: Mean Time Between Failures)
 - درصد موفقیت عملیات
- مثال:
- سامانه پرداخت باید با موفقیت ۹۹.۹٪ تراکنش ها را پردازش کند.
- نکته آموزشی:
- ممکن است سیستم در دسترس باشد (Available)، اما خروجی نادرست تولید کند یا عملکرد قابل اتکایی نداشته باشد (Unreliable)

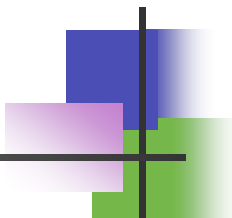
Capture NonFunctional Requirements(NFRs)

- **استحکام / تاب‌آوری (Robustness):** توانایی سیستم برای حفظ عملکرد صحیح در مواجهه با خطاها، داده‌های نامعتبر یا شرایط غیرمنتظره.
- سؤال کلیدی: اگر شرایط غیرعادی یا خطا رخ دهد، سیستم چه رفتاری از خود نشان می‌دهد؟
- مثال‌ها:
 - قطع اتصال اینترنت
 - ورود داده نامعتبر توسط کاربر
 - از کار افتادن یک سرویس خارجی
- نمونه : Requirement در صورت ناموفق بودن پرداخت بانکی، اطلاعات سفارش نباید از بین برود.
- تفاوت مهم:
- قابلیت اطمینان Reliability: میزان کم‌خطا بودن سیستم
- استحکام Robustness: میزان مقاومت سیستم در برابر خطاها و شرایط نامطلوب

Capture NonFunctional Requirements(NFRs)

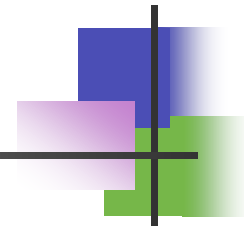
- هم‌کنش‌پذیری (Interoperability): توانایی سیستم در برقراری ارتباط، تبادل داده و تعامل با سایر سیستم‌ها.
- مثال‌ها:
 - اتصال به سامانه ERP
 - اتصال به درگاه‌های بانکی
 - یکپارچه‌سازی از طریق API
- نمونه Requirement: سامانه باید از REST API برای تبادل اطلاعات با سامانه‌های بیرونی استفاده کند.
- نکته: این ویژگی در سامانه‌های سازمانی و توزیع‌شده اهمیت بسیار بالایی دارد.

Capture NonFunctional Requirements(NFRs)



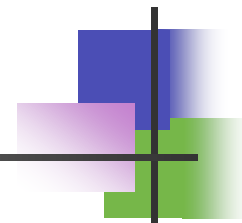
- قابلیت استفاده مجدد (Reusability) : توانایی استفاده مجدد از اجزاء، مؤلفه‌ها یا ماژول‌های سیستم در پروژه‌ها یا بخش‌های دیگر.
- در طراحی شی‌گرا اهمیت ویژه‌ای دارد:
- کلاس‌های قابل استفاده مجدد (Reusable Classes)
- طراحی مبتنی بر مؤلفه (Component-Based Design)
- مثال:
- ماژول احراز هویت باید در سایر سامانه‌های سازمان نیز قابل استفاده باشد.

Capture NonFunctional Requirements(NFRs)



- نگهداشت پذیری (Maintainability): میزان سهولت اصلاح، توسعه، نگهداری و رفع اشکال سیستم.
- سؤال کلیدی: اعمال تغییرات در سیستم تا چه اندازه آسان یا دشوار است؟
- نمونه Requirement: تغییر قوانین مالیاتی باید بدون تغییر ساختار اصلی سامانه امکان پذیر باشد.
- ارتباط مستقیم با کیفیت طراحی:
- Cohesion (انسجام)
- Coupling (کوپلینگ/وابستگی)
- Design Quality (کیفیت طراحی)

Capture NonFunctional Requirements(NFRs)



■ کارایی عملکردی(Performance): میزان سرعت، توان پاسخ‌دهی و ظرفیت پردازشی سیستم.

■ شاخص‌های متداول:

• زمان پاسخ (Response Time) = زمان دریافت پاسخ - زمان ارسال درخواست (مثلاً اگر کاربر دکمه «جستجو» را بزند و نتیجه پس از ۲ ثانیه نمایش داده شود)

• نرخ پردازش (Throughput): تعداد درخواست‌ها، تراکنش‌ها یا عملیات‌هایی که سیستم در یک بازه زمانی مشخص می‌تواند پردازش کند. (مثلاً یک سامانه بانکی قادر به پردازش ۱۰,۰۰۰ تراکنش در دقیقه باشد).

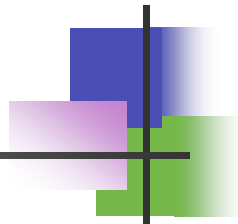
• تأخیر (Latency): مدت زمان تأخیر بین شروع یک رویداد و آغاز پاسخ سیستم؛ یعنی زمانی که طول می‌کشد تا اولین واکنش سیستم رخ دهد. Latency معمولاً بخشی از Response Time است، نه کل آن. (مثلاً کاربر درخواست می‌فرستد. یک ثانیه طول می‌کشد تا سرور پاسخ را شروع کند. و ۲ ثانیه دیگر طول می‌کشد تا کل پاسخ کامل شود).

■ نمونه نیازمندی‌ها: (نیازمندی‌های غیرعملکردی باید قابل اندازه‌گیری (Measurable) باشند)

✗ نامناسب: سیستم باید سریع باشد.

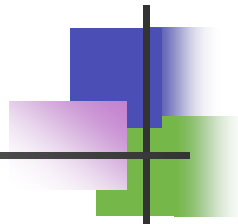
✓ مناسب و قابل اندازه‌گیری: زمان پاسخ عملیات جستجو نباید بیش از ۳ ثانیه باشد.

Capture NonFunctional Requirements(NFRs)



- قابلیت حمل (Portability): توانایی اجرای سیستم در محیط‌ها، سیستم‌عامل‌ها یا پلتفرم‌های مختلف.
 - مثال‌ها:
 - Windows / Linux
 - Mobile / Web
 - نمونه Requirement: سامانه باید روی مرورگرهای Chrome ، Firefox و Edge به درستی اجرا شود.

Capture NonFunctional Requirements(NFRs)



■ بهره‌وری منابع (Efficiency) : میزان استفاده بهینه سیستم از منابع محاسباتی و زیرساختی.

■ نمونه منابع:

• CPU

• Memory

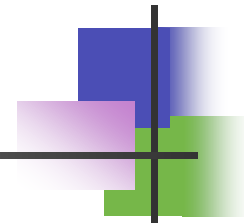
• Bandwidth

■ مثال Requirement : میزان مصرف حافظه سیستم نباید از ۵۰۰ MB فراتر رود.

Capture NonFunctional Requirements(NFRs)

- **نیازمندی‌های غیرعملکردی به عنوان محدودیت (Constraint):** نکته مهم آن است که بخشی از نیازمندی‌های غیرعملکردی (NFR) در عمل نقش **محدودیت (Constraint)** را ایفا می‌کند؛ یعنی شرایط و قیودی را بر طراحی و پیاده‌سازی سیستم تحمیل می‌کند.
 - محدودیت زمانی: پروژه باید ظرف ۴ ماه تحویل شود.
 - محدودیت فناوری: سامانه باید با Java پیاده‌سازی شود.
 - محدودیت محیطی: سامانه فقط روی شبکه داخلی سازمان اجرا شود.
 - محدودیت منابع: سیستم باید بر روی سرورهای موجود سازمان اجرا شود.
 - نکته مهندسی نیازمندی:
- هر Constraint یک NFR محسوب می‌شود، اما هر NFR الزاماً Constraint نیست.**

Capture NonFunctional Requirements(NFRs)



ارتباط نیازمندی‌های غیرعملکردی با **Use Case**: در این بخش معمولاً ابهام ایجاد می‌شود. به‌طور کلی دو حالت وجود دارد:

۱. حالت اول: **Special Requirements** که حالت یک نیازمندی غیرعملکردی است که وابسته به یک **Use Case** خاص است

- مثال: در یوزکیس لاگین یک نیازمندی (**Special Requirement**) داریم با عنوان

- فرآیند احراز هویت باید در کمتر از ۱ ثانیه انجام شود.

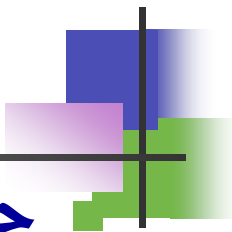
- رمز عبور کاربران باید با الگوریتم **AES-256** رمزنگاری شود.

۲. حالت دوم: **Supplementary Requirements** که حالت یک نیازمندی غیرعملکردی است که وابسته به هیچ **Use Case** خاصی نیست و اصطلاحاً سراسری (**Global**) است

- مثال: سامانه باید ۹۹.۹۹٪ در دسترس باشد.

- این **Requirement** به یک **Use Case** خاص محدود نیست، بلکه کل سامانه را تحت تأثیر قرار می‌دهد.

- چنین مواردی در سند **Supplementary Requirements** ثبت می‌شوند.



چهار مرحله اصلی برای بدست آوردن نیازمندیها

این چهار مرحله به ترتیب انجام میشود :

■ بدست آوردن لیست از نیازمندیهای اولیه

List candidate requirements

■ درک زمینه و بافت سیستم

Understand system context

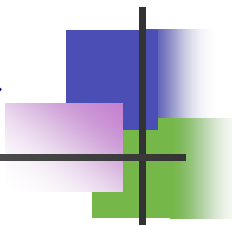
■ بدست آوردن نیازمندیهای کارکردی

Capture functional requirements

■ بدست آوردن نیازمندیهای غیرکارکردی

Capture nonfunctional requirements

پنج گام اصلی برای بدست آورد FR ها



1. شناسایی کنشگرها و یوزکیس ها

2. الویت دهی یوزکیس ها

3. اضافه کردن جزئیات به یوزکیس ها

4. پروتوتایپ واسط کاربر (UI)

5. ساختاردهی به یوزکیس مدل (روابط بین آکتورها و یوزکیس ها
تعریف و اضافه شوند)

Artifacts of requirements workflow: Use case model

هر یوزکیس مدل شامل بخش های زیر است.(محصولات این بخش)

آکتور یا کنشگر (Actor): کسی که از سیستم استفاده میکند و یا سیستمی خارجی که با سیستم تعامل دارد

اینکه چه سیستم با سیستم تعامل دارد بسته به محدود سیستم (system boundaries) دارد که در uml 2.0 به آن موضوع سیستم یا system subject میگویند

اگر یوزکیس در سطح زیرسیستم رسم شود(یعنی Subject یکی از زیرسیستم ها باشد)، ممکن است حتی زیر سیستم های دیگر بعنوان کنشگر آن سیستم باشند

حتی در سطح دیگر ریزدانگی؛ میتوان یک کلاس(پیچیده) را بعنوان subject در نظر گرفت که در این صورت کلاسهای دیگر که با این کلاس در تعاملند، بعنوان کنشگر در نظر گرفته میشوند(هر واحدی که عملکرد یا Functionality دارد میتواند Subject در نظر گرفته شود ولی بایستی ارزش این کار را داشته باشد، معمولاً کلاس در این سطح نیست)

یوزکیس ها (Use case) شامل:

Flow of events

: گامهای داخلی یوزکیس که براساس آن مشخص میشود که چه عملیاتی توسط کدام آکتورها انجام شود تا

یوزکیس محقق گردد.

Special requirements: نیازمندیهای غیرکارکردی خاص یوزکیس

Use Case Priorities: الویتهای براساس MoSCoW prioritization مشخص میشود

(won't have و must have/should have/ could have)

Glossary

اصطلاحات عمومی و مهمی که برای بیان توصیف سیستم و توضیحاتی در مورد سیستم توسط تیم تحلیل مورد استفاده قرار میگیرد.

User Interface Prototype: این پروتوتایپ برای ساختن ui در مراحل بعدی استفاده میشود و دور

ریختنی نیست .

۱. شناسایی کنشگرها و یوزکیس ها

اهداف

- شناسایی مرزهای **subject** (سیستم یا موضوع را از فضای بیرونی آن شناسایی کنیم)
- یوزکیس ها همواره داخل سیستم محسوب میشوند و کنشگرها بیرون
- چه افرادی و چه چیزهایی (سیستم هایی) با سیستم تعامل خواهند داشت (آکتورها) و چه انتظاراتی از سیستم دارند (یوزکیس)
- انواع کنشگر
 - عامل انسانی
 - سیستم - زیرسیستم یا **subject** های دیگر
 - وسیله یا **Device** سخت افزاری
- ساختن یک گلاسوری که تمام واژگان مورد نیاز در آن ثبت شوند (به گلاسوری موجود اضافه میشوند)

۱. شناسایی کنشگرها و یوزکیس ها

مراحل

- شناسایی کنشگرها (آکتورها)
- شناسایی یوزکیس ها
- توصیف خلاصه ای از هر یوزکیس
- مدل کردن آکتورها و یوزکیس ها (رسم دیاگرام یوزکیس)

۱. شناسایی کنشگرها و یوزکیس ها

شناسایی آکتور

- حداقل یک مصداق واقعی از آن بایستی وجود داشته باشد که با سیستم تعامل کند (آکتور یک نقش است و یک فرد نیست و یک فرد میتواند نقش های مختلف داشته باشد)
- حداقل همپوشانی آکتورهای مختلف (مطلوب است که همپوشانی نداشته باشند ولی امکان پذیر نیست)

۱. شناسایی کنشگرها و یوزکیس ها

شناسایی یوزکیس

- برای آکتور (کنشگر) بایستی یک نتیجه قابل مشاهده و واجد ارزش ایجاد کند
- برای کنشگر شروع کننده یوزکیس (initiating Actor) بطور معمول ذینفع اصلی یوزکیس است ولی سایر آکتورها نیز از آن استفاده میکنند
- یوزکیس بایستی اتمیک باشد (یا کل آن انجام میشود و یا اصلا انجام نمیشود) و نبایستی مرکب بوده و ترکیبی از چند کار مستقل باشد
- یوزکیس در قلمرو مسئله است و در رابطه با راه حل و نحوه پیاده سازی آن صحبت نمیشود (در قلمرو پاسخ و جواب یوزکیس نداریم)
- از یوزکیس های بسیار کوچک و بسیار بزرگ پرهیز شود. یوزکیس کوچک احتمال یک گام از یک یوزکیس باشد و یوزکیس بزرگ ترکیبی از چند یوزکیس است که بایستی به آنها شکسته شود.

۱. شناسایی کنشگرها و یوزکیس ها

توصیف مختصر هر یک از یوزکیس ها

بیان گامهای اصلی هر یک از یوزکیس ها

- مثال : مراحل برداشت پول از ATM (درخواست-کنترل موجودی- کسر از حساب-پرداخت-چاپ رسید)
- شامل حالات خاص نمیشود
- شامل جزئیات نمیشود

رسم اولیه دیاگرام یوزکیس

- آکتورهای اولیه و آکتورهای ثانویه تعریف میشوند و رابطه آنها با یوزکیس تعریف میشوند
- کنشگر اولیه و ثانویه نسبت به یک یوزکیس مشخص میشود و نسبی است ولی ممکن است یک کنشگر ثانویه در هیچ یوزکیسی اولیه نشود (اجباری نیست)
- این نمودار اولیه است و پایه اصلی نمودار یوزکیس است که در مراحل بعدی ارتباطات **include, extend, generalization** و.... به آن اضافه میشود.
- این نمودار برای مشتری و کاربر قابل فهم است. و آنها بایستی با مرور اطلاعات نسبت به تایید آن اقدام نمایند.
- **Time Actor**: برای عملیتهای دوره ای که بطور خودکار انجام میشود، یک آکتور جعلی ساخته میشود که

۱. شناسایی کنشگرها و یوزکیس ها

اولویت بندی یوزکیس ها

- براساس این اولویتها مشخص میکنیم کدام یوزکیس بایستی سریعتر پیاده سازی شوند و کدامیک برای مراحل بعدی بمانند
- در صورت کمبود منابع نظیر زمان، مشخص میشود کدام یوزکیس ها میتوانند در مراحل بعدی پیاده سازی شوند
- رعایت MoSCoW: اولویت بندی براساس دید مشتری (user value)
 - **Must Have**: اگر پیاده سازی نشود، پروژه شکست خورده است
 - **Should Have**: خیلی مهم است که پیاده سازی شود ولی اگر نشد پروژه شکست نخورده است
 - **Could Have**: اگر منابع مازاد داشتیم پیاده سازی میشوند
 - **Wont Have**: پیاده سازی نمیشوند
- در الویت بندی ممکن است **Wont have** به **Should have** تبدیل شود.
- رسیدن به امور دارای ریسک (یوزکیس های دارای ریسک) الویت دارند. بنابراین اگر دو یوزکیس از دید **Moscow** هم الویت باشند، الویت با یوزکیسی است که ریسک بالاتری دارد.
- روابط پیش نیازی الویتها روی الویت یوزکیس تاثیر گذار است
 - یک یوزکیس **Could have** پیشنهاد یک یوزکیس **Should have** است، این دیگر **Could have** نیست و بایستی زودتر از یوزکیس وابسته انجام شود.

الویت دهی بر اساس مسکو



۲. اولویت بندی یوزکیس ها

■ براساس این اولویتها مشخص میکنیم کدام یوزکیس بایستی سریعتر پیاده سازی شوند و کدامیک برای مراحل بعدی بمانند

■ در صورت کمبود منابع نظیر زمان، مشخص میشود کدام یوزکیس ها میتوانند در مراحل بعدی پیاده سازی شوند

■ رعایت MoSCoW: اولویت بندی براساس دید مشتری (user value)

■ **Must Have**: اگر پیاده سازی نشود، پروژه شکست خورده است

■ **Should Have**: خیلی مهم است که پیاده سازی شود ولی اگر نشد پروژه شکست نخورده است

■ **Could Have**: اگر منابع مازاد داشتیم پیاده سازی میشوند

■ **Wont Have**: پیاده سازی نمیشوند

■ در الویت بندی ممکن است **Wont have** به **Should have** تبدیل شود.

■ رسیدن به امور دارای ریسک (یوزکیس های دارای ریسک) الویت دارند. بنابراین اگر دو یوزکیس از دید **Moscow** هم الویت باشند، الویت با یوزکیسی است که ریسک بالاتری دارد.

■ روابط پیش نیازی الویتها روی الویت یوزکیس تاثیر گذار است

■ یک یوزکیس **Could have** پیشنیاز یک یوزکیس **Should have** است، این دیگر **Could have** نیست و بایستی زودتر از یوزکیس وابسته انجام شود.

۳. اضافه کردن جزئیات به یوزکیس

برای هر یوزکیس گامهای یوزکیس که به عنوان گامهای اصلی یا جریان وقایع (Flow of Events) شناخته میشود، توصیف شود

- در یوزکیس جزئیات وابسته به پیاده سازی بیان نمیشود
- یوزکیس در فضای مسئله است و دید تحلیل به مسئله دارد. بنابراین از بیان عباراتی نظیر با فشار دادن دکمه X به صفحه Y میرویم و....؛ پرهیز میشود (جزئیات پیاده سازی است)
- گامهای یوزکیس بیان نمیشود.
- گام شروع یوزکیس (Start State) که الزاما توسط آکتور اصلی برداشته میشود به گام تریگر معروف است.
- هر گام توسط یک آکتور و یا سیستم انجام میشود.

۳. اضافه کردن جزئیات به یوزکیس

بیان گامها بصورت زبان ساختیافته (انگلیسی، فارسی یا هر زبان دیگر)

- **Basic Path:** مسیر اصلی پردازش که به خاتمه موفق می انجامد.
- **Alternative Path:** سایر مسیرهای یوزکیس که خاتمه دیگری دارد.
- برای هر یوزکیس، هم مسیر اصلی و هم مسیرهای جایگزین بیان میشوند ولی در این مرحله فقط مسیر اصلی مورد نظر است
- برای بیان دقیق یوزکیس از ساختار جدولی استفاده میشود که شرایط (پیش از شروع و شرط خاتمه یا **Post Condition**) به همراه مسیرهای اصلی، جایگزین، کنشگرهای اصلی و جایگزین و... آورده میشود
- ز

۳. اضافه کردن جزئیات به یوزکیس

چه چیزهای در توصیف یک یوزکیس آورده میشود

- وضعیت شروع و خاتمه در قالب **Pre Condition** و **Post Condition** بیان میشود
- چگونه و چه زمانی یوزکیس شروع میشود و خاتمه می یابد
- ترتیب عملیات مورد نیاز
- مسیرهایی که استفاده از آنها غیرمجاز است (و بخشی از یوزکیس نیستند و این مسیر را قبلا رفته و متوجه اشتباه شده ایم)، نیز ثبت میشود که به عنوان تجربه نگهداری شود
- مسیرهای جایگزین نیز توصیف میشود
- در هر گام، نحوه تعامل آکتور با سیستم بیان میشود (چستی کار بیان میشود، چگونگی در فضای راه حل است) - چه کاری توسط آکتور انجام میشود و چه کاری توسط سیستم
- ممکن است در حین عملیات (یک مرحله) از مقادیر، اشیاء و منابع درونی سیستم استفاده شود (مثلا استفاده از اسکناس شمار در سیستم **ATM** و در مرحله تحویل پول)
- محدودیتهایی که کاربر روی سامانه می آورد، نظیر استفاده از یک پلات فرم خاص، سخت افزار ویژه و.... (که بایستی در فضای راه حل بیان شود)، را میتوان در فضای مسئله بیان نمود و میتوان در **use case** نیز مدل کرد. برخی تحلیل گرها با انجام این کار بشدت مخالف هستند زیرا این کار باعث میشود که مدل تحلیل وابسته به پیاده سازی شود و عملا مدل قابلیت استفاده مجدد خود را از دست می دهد.

۳. اضافه کردن جزئیات به یوزکیس

■ فرموله کردن توصیف یوزکیس

- این عمل بصورت متنی و در ساختار جدولی بیان میشود
- اگر یوزکیس پیچیده باشد از نمودارهایی نظیر **Activity Diagram** استفاده میشود
- امکان استفاده از دیاگرامهای دیگر نیز وجود دارد که باعث میشود دیدی خاص از نحوه
- از **State Chart Diagram** استفاده نمود و حالتها را بیان نمود
- استفاده از **interaction diagram** ها نظیر **collaboration** و یا **Sequence Diagram**
- مهم ترین و رایج ترین ابزار در این مرحله استفاده از **Activity Diagram** است

۳. اضافه کردن جزئیات به یوزکیس

فرموله کردن توصیف یوزکیس (چند نکته مهم)

- **Use case** دید شی گرایي ندارد و **Sequence diagram** کاملاً شی گرا و بیان میکند یک یوزکیس چگونه بواسط اشیاء سیستم محقق میشود. مراحل زیر را انجام میدهیم (یک درز پارادایمی یا **Seam Paradigm** وجود دارد)

- برای پوشش این درز از **Activity diagram (Swim Laned)** استفاده میشود

- اول یوزکیس با **Activity** بیان میشود

- **Swim lane** ها یا **Actor** هستند و یا سیستم؛ عملکرد سیستم بواسط **Swim Lane** های سیستم بیان میشود بنابراین **Swim Lane** های سیستم شکسته میشود به **Swim Lane** های کوچکتری که داخل هر کدام یک **object** قرار دارد. در نتیجه میتوان گفت داخل **Activity diagram** کار چگونه توسط اشیاء انجام میشود

- سپس **Swim Lane** سیستمی شکسته شده را با استفاده از **Sequence Diagram** مدل میکنند

- بطور معمول برای تمام یوزکیس ها (حتی یوزکیس های ساده و غیر پیچیده) **Activity Diagram** رسم میشود.

- بطور معمول **Activity** در مرحله تحلیل رسم میشود (در حال حاضر در مرحله جمع آوری نیازمندیها هستیم)

- در مرحله تحلیل بیان میشود با استفاده از کلاسهای قلمرو مسئله چگونه **use case** محقق میشود.

۴. پروتوتایپ کردن واسط کاربر

■ دلیل ؟

- در بسیاری از سیستم ها، نیازمندیها از طریق UI قابل مشاهده است. مثلاً در یک سیستم با بررسی منوها و زیرمنوها، یوزکیس ها قابل شناسایی است.
- بعضی سیستم های تجاری، عملکردشان از طریق منوها و زیرمنوها مشخص است و این کمک میکند که نیازمندیهای جدید شناسایی شود و بعضی از سیستم ها این ویژگی را ندارد (سیستم هایی که عملکردشان برای انجام تنظیمات است مثل **Event Driven** مناسب برای این کار نیست)
- **UI** در قلمرو جواب و حل مسئله است ولی در این مرحله برای شناسایی نیازمندیهاست
- این نسخه **UI** اولیه بوده و فقط برای تعیین بهتر قلمرو مسئله (مشتری و کاربر از طریق یک محصول ملموس میتواند بازخورد بهتری داشته باشد) است.
- این پروتوتایپ اولیه است و بطور معمول قابل استفاده در سایر مراحل نیست.

۴. پروتوتایپ کردن واسط کاربر

■ ساخت یک واسط کاربری بصورت منطقی

- یعنی بیان میشود که چه اجزا و المانهایی برای UI لازم است تا آکتور بتواند سرویس مورد نظر را از یوزکیس بخواهد
- مثلاً آکتور بتواند از داخل یک لیست، کالای مورد نیاز خود را انتخاب کند و به لیست کالاهای انتخاب شده خود منتقل کند. این متن براحتی بیانگر المانهای UI است
- ارتباط بین المانها بیان شود
- شکل ظاهری المانها
- نحوه عملیات از طریق آنها
- لزومی ندارد در این مرحله عملیات از طریق ابزارهای پروتوتایپ پیاده سازی شود حتی میتوان از وایت برد و **sticky note** هم استفاده نمود.
- المانها ابزار اولیه هستند و وابسته به هیچ تکنولوژی خاصی نیستند. از اصطلاحاتی نظیر کمبوباکس و یا ویجت های خاص استفاده نمیکنیم

۴. پروتوتایپ کردن واسط کاربر

■ ساخت یک واسط کاربری بصورت فیزیکی

- وارد فضای ابزار و تکنولوژی میشویم
- مجموعه المانهایی که تشکیل دهنده صفحات مرتبط با یک اکتور شناسایی شده، طراح میشود و سپس با استفاده از یک ابزار پیاده سازی میشود
- اضافه کردن منو و زیر منوها و...
- تک تک صفحات بصورت مجزا طراحی میشوند
- استفاده از چارت محاوره برای بیان منو، سیم بندی صفحه و نحوه حرکت از یک صفحه به صفحه دیگر و... (ناوش)
- هر اکتور بایستی تعریف ساده ای از صفحات داشته باشد تا بتوان یوزکیس های خود را بخوبی انجام دهد.
- برای انجام یک کار (یا کارهای مشابه) از روشهای مختلف استفاده نشود (UI را پیچیده نکنید)
- پروتوتایپ برای تایید در اختیار کاربر قرار میگیرد.

۵. ساختار دادن به مدل یوزکیس و رسم آن

■ مشخص کردن روابط بین یوزکیس ها

■ روابط Gen./Spec

■ روابط بین یوزکیس ها را مشخص میکنیم

■ Extend

■ Include